Lecture 10. Introduction to Circuit Complexity and Polynomial Method

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Boolean circuits

- A simplified model of real circuits
- Easier to prove lower bounds



Boolean circuits

- A simplified model of real circuits
- Easier to prove lower bounds

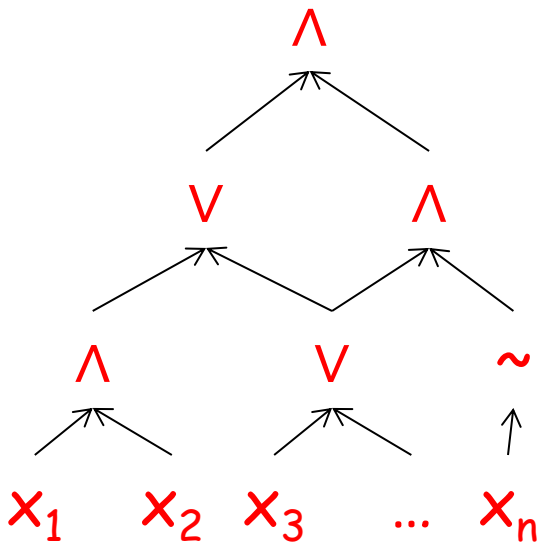
Does **NP** have linear-size, log-depth Boolean circuits ??





Boolean circuits

- **circuit** C
  - directed acyclic graph
  - nodes: AND ($\wedge$); OR ($\vee$); NOT ($\sim$); variables x_i

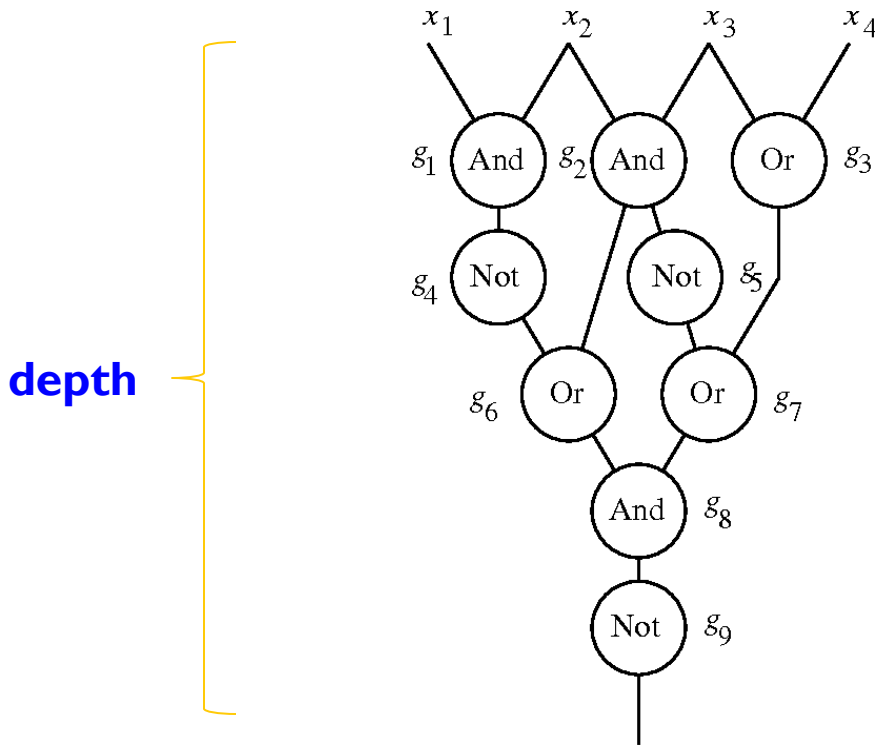


- C computes function $f:\{0,1\}^n \rightarrow \{0,1\}$ in natural way
  - identify C with function f it computes



Boolean circuits

- **size** = # gates
- **depth** = longest path from input to output



Boolean circuits

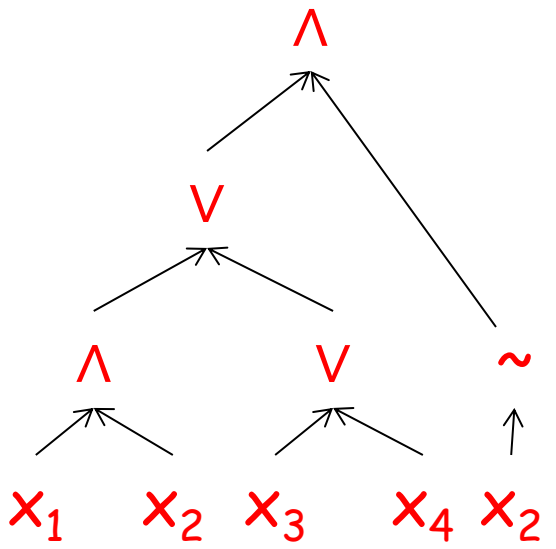
- **size** = # gates
- **depth** = longest path from input to output

- every function $f:\{0,1\}^n \to \{0,1\}$ is computable by a circuit of size at most $O(n2^n)$
  - AND of n literals for each x such that f(x) = 1
  - OR of up to 2^n such terms

  - but superexponential time complexities exist, such as $2^{2^{\mathrm{poly}(n)}}$



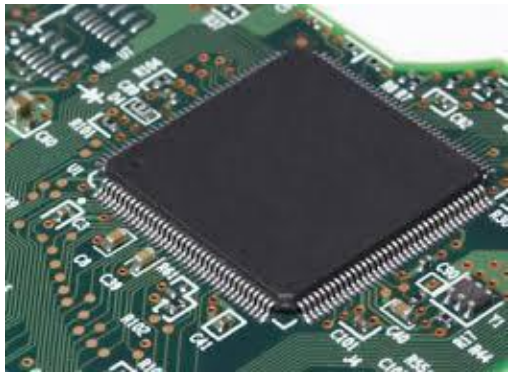
- **formula (or expression)**: graph is a tree
  - $((x_1 \wedge x_2) \vee (x_3 \vee x_4)) \wedge (\sim x_2)$



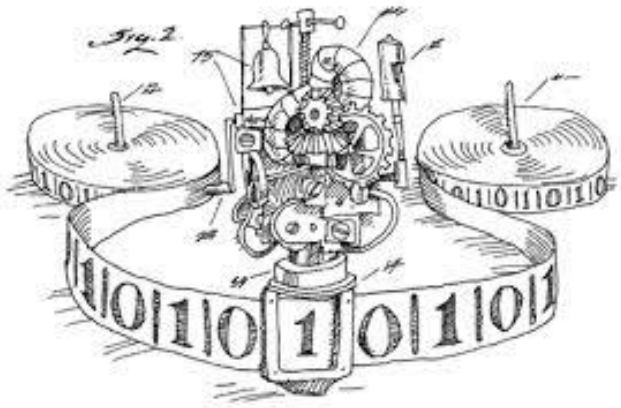


Difference between circuit and TM?

- circuit only works for a specific input length



  - different from Turing machine!





Circuit families

- circuit only works for a specific input length
- we're used to functions $f:\sum^* \rightarrow \{0,1\}$ for TMs
- circuit **family** : a circuit for each input length $C_1, C_2, C_3, \ldots$ = "$\{C_n\}$"
- "$\{C_n\}$ computes f" iff for all x

$$C_{|x|}(x) = f(x)$$

- "$\{C_n\}$ decides L", where L is the language associated with f



P/poly

- T(n)-size circuit family:
  - a sequence $\{C_n\}$ of Boolean circuits where the size of C_n is $\leq T(n)$ for every n
- SIZE(T(n))
  - the languages decided by T(n)-size circuit families $\{C_n\}$
- P/poly
  - the class of languages decided by polynomial-sized circuit family
  - P/poly= $\cup_c SIZE(n^c)$



Connection to TMs

- Is $P \subseteq P/poly$?



Connection to TMs

- Is $P \subseteq P/poly$?

- Boolean circuit is equivalent to "straight line programs":
  - no branching, loop, goto, etc.

- So we need to change any Turing Machine to such a straight line program when fixing the input length



$P \subseteq P/poly$

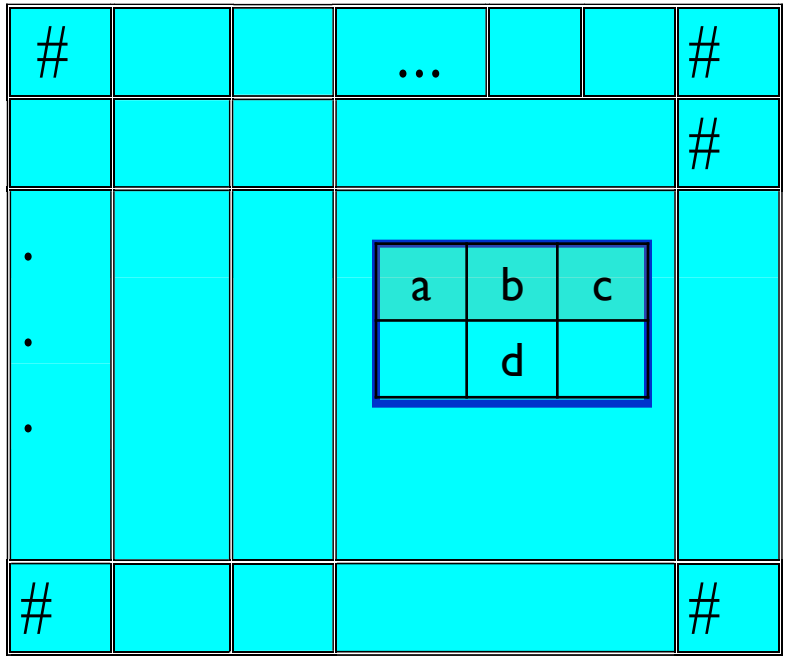
- Suppose a TM M running in time t(n) decides language L, we can build circuit family $\{C_n\}$ of size $|C_n| = O(t(n)^2)$ that decides L
  - Proof: by computation history

- Conclude: $L \in \mathbf{P}$ implies family of polynomial-size circuits that decides L



Proof of P ⊆ P/poly

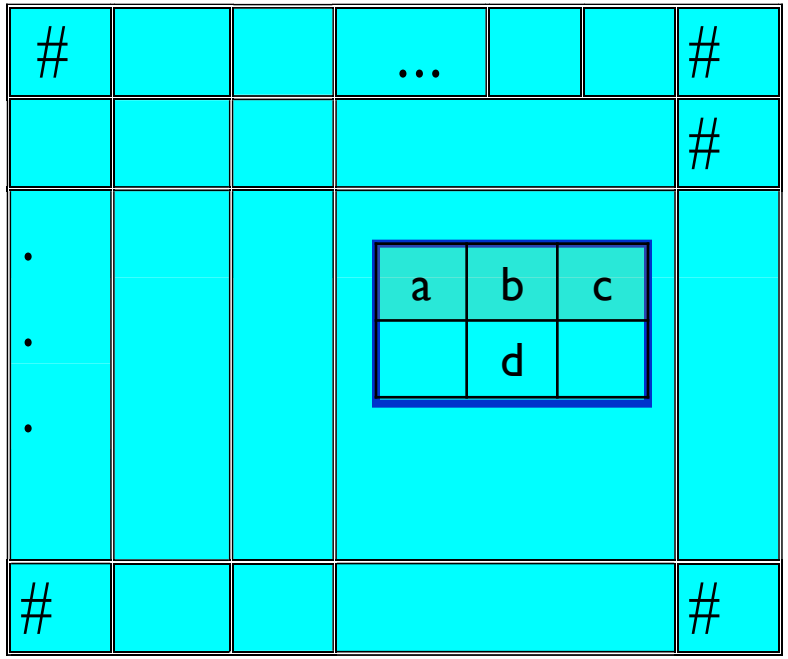
- Suppose a TM M of t(n) time decides L
- Consider the t(n)×t(n) table of computation history:
- We want to use a circuit to generate it
  - each d can be decided by a,b,c





- Γ is the set of tape symbols, Q is the set of states, so the possible elements of entry (i,j) in the table is in Γ∪Q
- We assign each entry (i,j) |Γ∪Q| Boolean variables:
  - v(i,j,a)=1 iff the entry (i,j) in the table is a
  - otherwise v(i,j,a)=0

•Then, construct the entire circuit by v(i,j,a) for all i,j∈[1,t(n)] and a∈Γ∪Q
•From the first line, we can compute each line one by one



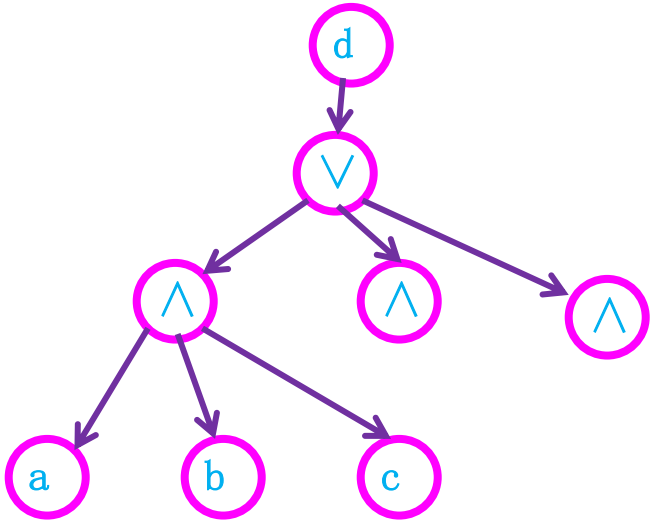
a	b	c
	d	

- In the circuit, for every variable v(i,j,d):

  $v(i,j,d)=\bigvee_{(a,b,c) \text{ yields } d}(v(i-1,j-1,a) \wedge v(i-1,j,b) \wedge v(i-1,j+1,c))$

- For the last line, check whether it has the accept state

- So the circuit complexity for a TM in t(n) time is $O(t(n)^2)$
- $P \subseteq P/poly$





Connection to TMs

- other direction? (P/Poly $\subseteq$ P ?)

- any ideas?



Connection to TMs

- other direction? (P/Poly ⊆ P ?)

- A poly-size circuit family:
  - $C_n = x_1 \wedge x_2 \wedge \ldots \wedge x_n$ if M_n halts
  - $C_n = 0$ if M_n loops

- decides (unary version of) HALT!
  - $\{1^n \mid n$ is the code of a TM with no input which halts$\}$
- Even *an undecidable language* can be in P/poly



Uniformity

- Strange aspect of circuit family:
  - can "encode" (potentially uncomputable) information in family specification
- solution: uniformity – requires specification is simple to compute
  - Definition: circuit family $\{C_n\}$ is **logspace uniform** iff a TM M outputs C_n on input 1^n and runs in O(log n) space



Uniformity

Theorem: **P** = languages decidable by logspace uniform, polynomial-size circuit families $\{C_n\}$

- Proof:
  - ($\Rightarrow$) already saw $P \subseteq P/poly$, you can check it is logspace uniform
  - ($\Leftarrow$) on input x, generate $C_{|x|}$, evaluate it and accept iff output=1



TMs that take advice

- family $\{C_n\}$ without uniformity constraint is called "non-uniform"
- regard "non-uniformity" as a limited resource just like time, space, as follows:
  - add read-only "advice" tape to TM M
  - "advice": a sequence {A(n)}, only depends on |x|
  - M "decides L with advice A(n)" iff

$$M(x, A(|x|)) \text{ accepts} \Leftrightarrow x \in L$$



TMs that take advice

- Definition: **TIME(t(n))/f(n)** = the set of those languages L for which:
  - there exists A(n) s.t. |A(n)| = O(f(n))
  - TM M decides L with advice A(n) in time t(n)
- most important such class:

$$\mathbf{P/poly} = \cup_k \mathbf{TIME}(n^k)/n^k$$



Approach to P/NP

- Believe **NP** $\not\subseteq$ **P**
  - equivalent: "**NP** does not have uniform, polynomial-size circuits"
- *Even believe* **NP** $\not\subseteq$ **P/poly**
  - equivalent: "**NP** (or, e.g. SAT) does not have polynomial-size circuits"
  - implies **P** $\neq$ **NP** (since P $\subseteq$ P/poly)
  - many believe: best hope for **P** $\neq$ **NP**

- Of course, NP $\subseteq$ P/poly doesn't imply P=NP



Karp-Lipton

Theorem (KL): if SAT has poly-size circuits then **PH** collapses to the second level

- Proof:
  - suffices to show $\Pi_2 \subseteq \Sigma_2$
  - $L \in \Pi_2$ implies L expressible as:

$$L = \{ x : \forall\, y\ \exists\, z\ (x, y, z) \in R \}$$

with $R \in \mathbf{P}$. *(polynomial on length of x)*



Karp-Lipton

$$L = \{ x : \forall y \; \exists z \; (x, y, z) \in R \}$$

- given (x, y), "$\exists z$ (x, y, z) $\in$ R?" is in **NP**
- *pretend* circuit C solves SAT,
- Claim: if SAT $\in$ P/poly, then L =

  { x : $\exists$C $\forall$y

  [use C to find z for "$\exists z$ (x, y, z) $\in$ R?", accept iff (x, y, z) $\in$ R]

  }

- size of C is also polynomial on |x|
- so it takes polynomial time to simulate C
- then verify z

poly time



Karp-Lipton

$$L = \{ x : \forall y \; \exists z \; (x, y, z) \in R \}$$

- given (x, y), "$\exists z$ (x, y, z) $\in$ R?" is in **NP**
- *pretend* circuit C solves SAT,
- Claim: if SAT $\in$ P/poly, then L =

  { x : $\exists$C $\forall$y

  [use C to find z for "$\exists z$ (x, y, z) $\in$ R?", accept iff (x, y, z) $\in$ R]

  }

poly time

- x $\in$ L:
  - some C decides SAT $\Rightarrow$ $\exists$C $\forall$y […] accepts
- x $\notin$ L:
  - $\exists$y $\forall$z (x, y, z) $\notin$ R $\Rightarrow$ $\forall$C $\exists$y […] rejects



Shannon's counting argument

- frustrating fact: *almost all* functions require huge **circuits**

Theorem (Shannon): With probability at least $1 - o(1)$, a random function

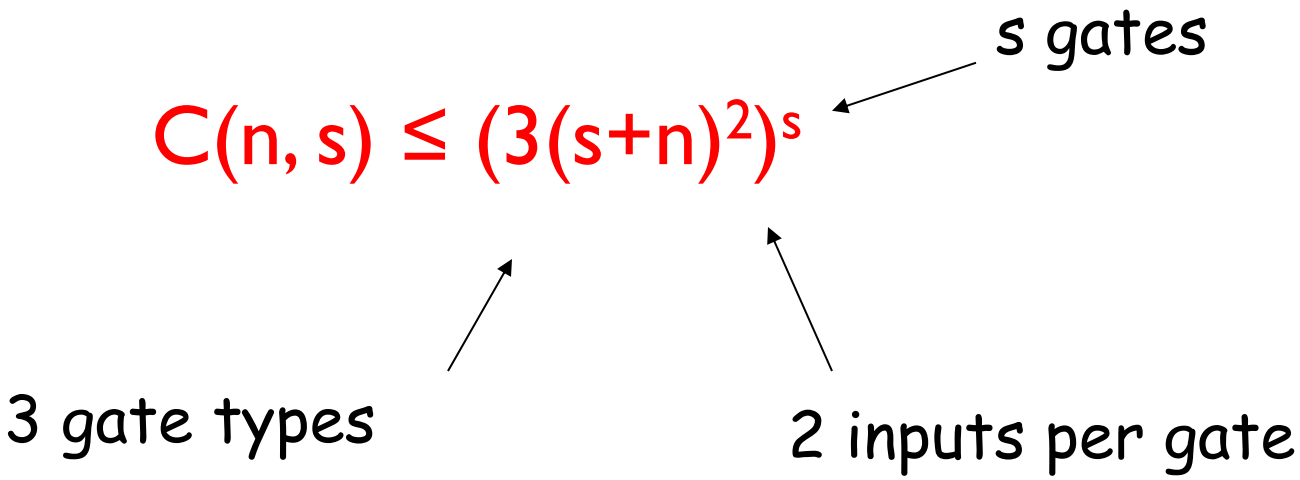
$$f:\{0,1\}^n \to \{0,1\}$$

requires a **circuit** of size $\Omega(2^n/n)$



Shannon's counting argument

- Proof (counting):
  - $B(n) = 2^{2^n}$ = # functions $f:\{0,1\}^n \rightarrow \{0,1\}$
  - # circuits with n inputs and size s, is at most





Shannon's counting argument

$$C(n, s) \leq (3(s+n)^2)^s$$

- $C(n, c2^n/n) < (c'2^{2n}/n^2)^{(c2^n/n)}$
  $< O(1/n^2)2^{2c2^n}$
  $< o(1)2^{2^n}$

  (choose $c<1/2$)

- probability that a random function has a circuit of size $s = o(2^n/n)$ is at most

$$C(n, s)/\ 2^{2^n} < o(1)$$



Hierarchy theorem for non-uniform circuit families

- with probability at least $1 - o(1)$, a random function $f:\{0,1\}^n \to \{0,1\}$ requires **circuit** of size $\Omega(2^n/n)$
- every function $f:\{0,1\}^n \to \{0,1\}$ is computable by a circuit of size at most $O(n2^n)$

For function T, T' with $2^n/(100n) > T'(n) > T(n) > n$ and $T(n)\log T(n) = o(T'(n))$,

$$\mathrm{SIZE}(T(n)) \subsetneq \mathrm{SIZE}(T'(n))$$



Hierarchy theorem for non-uniform circuit families

- There exists a function $f:\{0,1\}^n \to \{0,1\}$ not computable by **circuit** of size $2^n/(10n)$
- f is computable by a circuit of size at most $10n2^n$

- let $n = 1.1 \cdot \log N$, and $g:\{0,1\}^N \to \{0,1\}$ be the function which applies f on the first n bits

$$g \in \text{SIZE}(10n2^n) = \text{SIZE}(11 \cdot N^{1.1} \log N) \subseteq \text{SIZE}(N^2)$$

$$g \notin \text{SIZE}(2^n/(10n)) = \text{SIZE}(N^{1.1}/(11 \cdot \log N))$$

$$g \notin \text{SIZE}(N)$$



Relation to complexities classes

- We still cannot find any NP problem that can be proved to have no polynomial size circuit

- What we can get:

  $\mathrm{EXPSPACE} \not\subset \mathrm{P/poly}$



Relation to complexities classes

- What we can get:

  $$\text{EXPSPACE} \not\subset \text{P/poly}$$

- for every input length n, we find the "first" function f_n which needs circuit of size $\Omega(2^n/n)$

- Then the language $L=\cup_n\{x\in\{0,1\}^n: f_n(x)=1\}$ is not in P/poly
  - Why is it in EXPSPACE?



Relation to complexities classes

- What we can get:

  $$\text{EXPSPACE} \not\subset \text{P/poly}$$

- for every input length n, we find the "first" function f_n which needs circuit of size $\Omega(2^n/n)$
  - enumerating all functions $\{0,1\}^n \to \{0,1\}$ takes 2^n space
  - then enumerate all circuits of size $O(2^n/n)$
  - and all inputs from $\{0,1\}^n$

- Then the language $L=\cup_n\{x\in\{0,1\}^n: f_n(x)=1\}$ is not in P/poly
  - But it is in EXPSPACE



- Further, we have:

$$EXP^{\Sigma_2} \not\subset P/poly$$

- Why?



- Further, we have:

$$EXP^{\Sigma_2} \not\subset P/poly$$

- Similarly, we can find the "first" function f_n which needs circuit of size $\Omega(2^n/n)$
  - Given a function f (whole size 2^n), it's an NP problem to find an $O(2^n/n)$-size circuit that computes it
  - Since given a circuit C, check $f(x)=C(x)$ for all $x\in\{0,1\}^n$ is polynomial in the "size" of f



- Further, we have:

$$EXP^{\Sigma_2} \not\subset P/poly$$

- Similarly, we can find the "first" function f_n which needs circuit of size $\Omega(2^n/n)$
  - Given a function f (whole size 2^n), it's a NP problem to find an $O(2^n/n)$-size circuit that computes it. So given a NP oracle, we can check that all $O(2^n/n)$-size circuits cannot compute it
  - Given a function "range" $[f_1,f_2]$, By a $\Sigma_2=NP^{NP}$ oracle, we can check whether there is a function f that needs $\Omega(2^n/n)$ size circuit in the range

    $(\exists f\in[f_1,f_2])(\forall C$ of size $O(2^n/n))(C$ does not compute $f)$





- Further, we have:

$$EXP^{\Sigma_2} \not\subset P/poly$$

- Similarly, we can find the "first" function f_n which needs circuit of size $\Omega(2^n/n)$
  - Given a function f (whole size 2^n), it's a NP problem to find an $O(2^n/n)$-size circuit that computes it. So given a NP oracle, we can check that all $O(2^n/n)$-size circuits cannot compute it.
  - Given a function "range" $[f_1,f_2]$, By a $\Sigma_2=NP^{NP}$ oracle, we can check whether there is a function f that needs $\Omega(2^n/n)$ size circuit in the range.
  - A binary search over all 2^{2^n} functions with input length n takes 2^n time





- We also have:

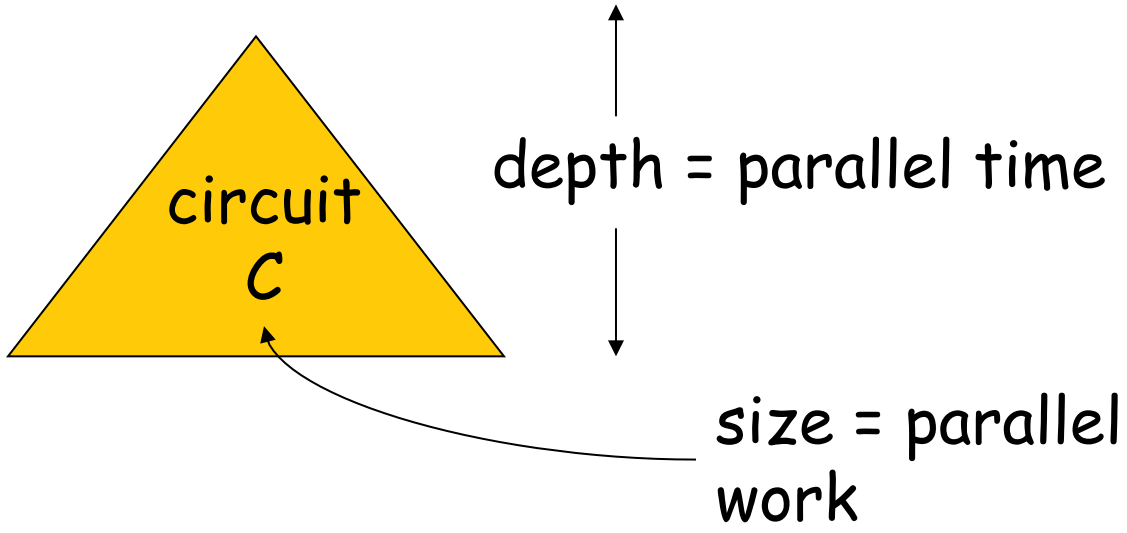
$$\mathrm{NEXP}^{\mathrm{NP}} \not\subset \mathrm{P/poly}$$

- proof by contradiction:
  - if $\mathrm{NEXP}^{\mathrm{NP}} \subset \mathrm{P/poly}$, then certainly $\mathrm{NP} \subset \mathrm{P/poly}$, by Karp-Lipton Theorem, PH collapses to $\boldsymbol{\Sigma}_2$
  - in particular, $P^{\Sigma_2} = \Sigma_2 = NP^{NP}$
  - by padding argument, $EXP^{\Sigma_2} = NEXP^{NP}$, but we already know $EXP^{\Sigma_2} \not\subset P/poly$



Parallelism

- uniform circuits allow refinement of polynomial time:





Parallelism

- the **NC** ("**N**ick's **C**lass" after Nick Pippenger) Hierarchy

(Note: here we require NC circuits be logspace uniform, but in many references there are no such restriction)

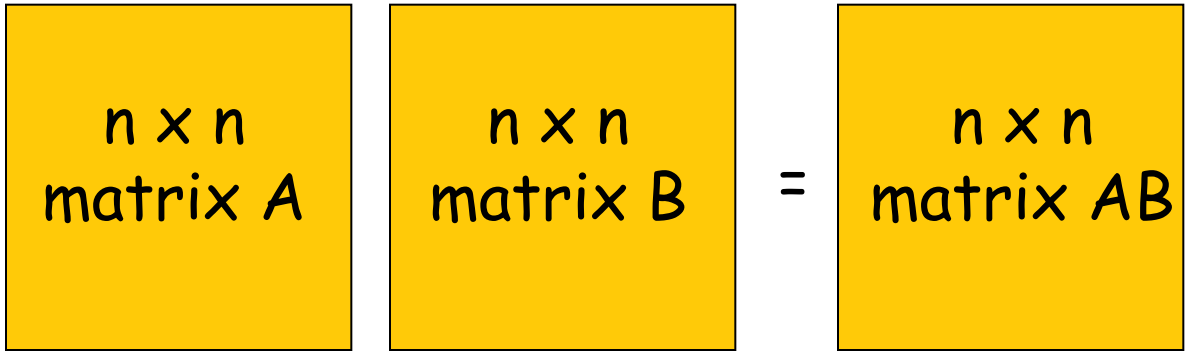
$$\mathbf{NC}^k = O(\log^k n) \text{ depth, poly}(n) \text{ size}$$

$$\mathbf{NC} = \bigcup_k \mathbf{NC}^k$$

- captures "efficiently parallelizable problems"
- not realistic? overly generous
- OK for proving non-parallelizable



Boolean Matrix Multiplication



- what is the parallel complexity of this problem?
  - size = poly(n)
  - depth = $\log^k(n)$? (which k?)



Boolean Matrix Multiplication

- two details
  - arithmetic matrix multiplication…

$$A = (a_{i,k})\ B = (b_{k,j}) \quad (AB)_{i,j} = \Sigma_k\ (a_{i,k} \times b_{k,j})$$

  … vs. Boolean matrix multiplication:

$$A = (a_{i,k})\ B = (b_{k,j}) \quad (AB)_{i,j} = \bigvee_k\ (a_{i,k} \wedge b_{k,j})$$

  - single output bit: to make matrix multiplication a language: on input A, B, i, j output $(AB)_{i,j}$



Boolean Matrix Multiplication

- Boolean Matrix Multiplication is in **NC^1**
  - level 1: compute n ANDS: $a_{i,k} \wedge b_{k,j}$
  - next log n levels: tree of ORs

  - n^2 subtrees for all pairs (i, j)
  - select correct one and output

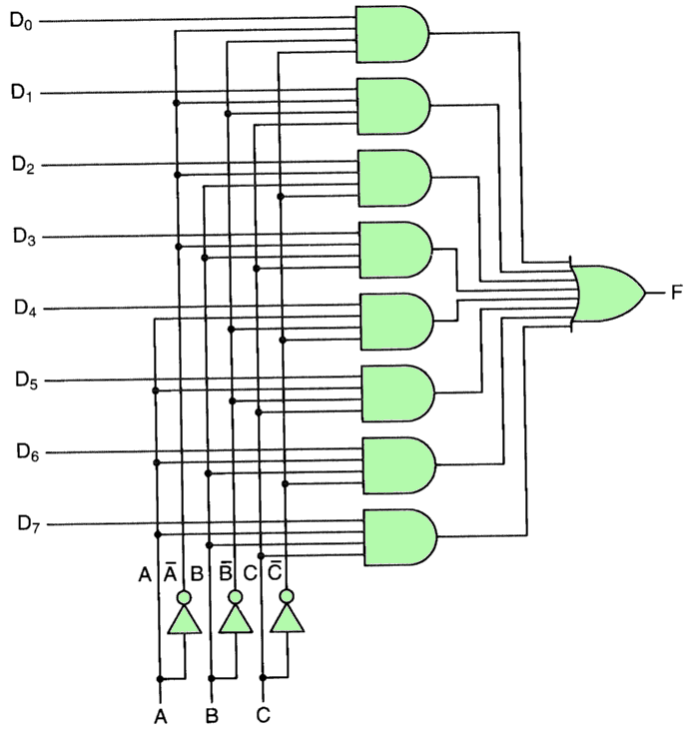


Multiplexers

- Basic concept
  - 2^k data inputs; k control inputs ("selects"); 1 output
  - Connects one of 2^k inputs to the output
  - "Selects" decide which input connects to output

A	B	C	OUTPUT
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

n



log n

$\mathbf{NC^1}$ and Boolean formulas

- Previous circuit is actually a formula. This is no accident:

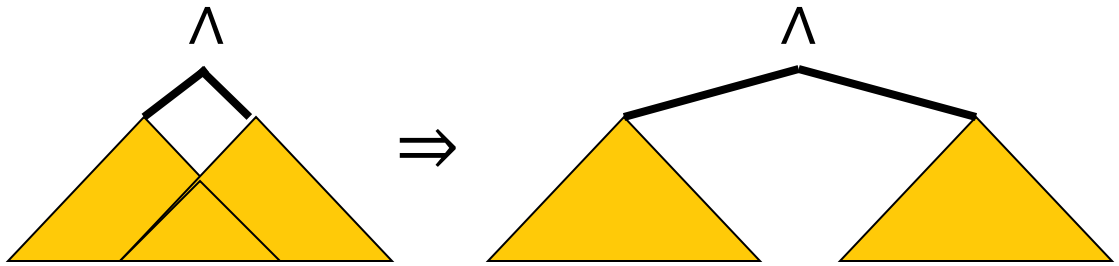
Theorem: $L \in \mathbf{NC^1}$ iff decidable by polynomial-size uniform family of Boolean formulas

Note: remember that formula is a circuit whose graph is a tree, and we measure formula size by leaf-size



$\mathbf{NC^1}$ and Boolean formulas

- Proof:
  - (⇒) convert $\mathbf{NC^1}$ circuit into formula
    - recursively:

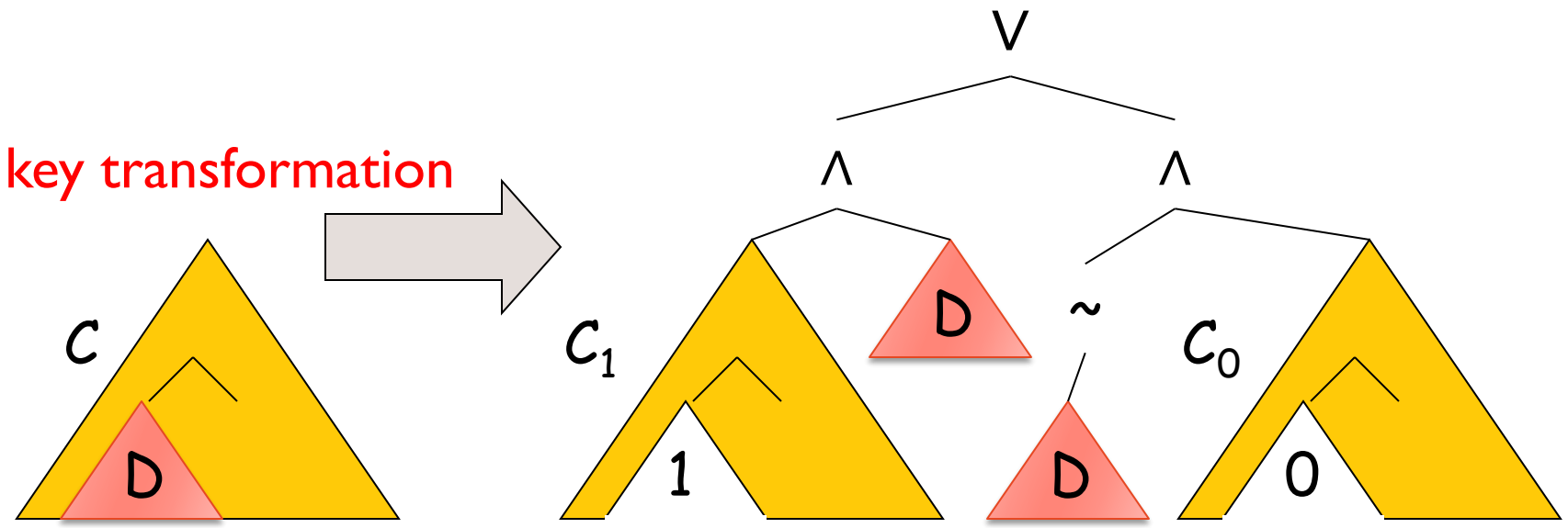


    - note: #nodes in a tree bounded by $2^{O(\log n)}$=poly(n)



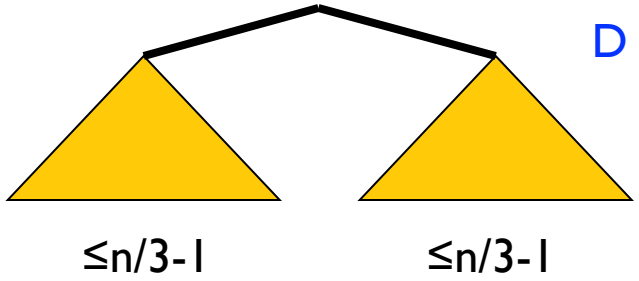
$\mathbf{NC^1}$ and Boolean formulas

- ($\Leftarrow$) convert formula of size n into formula of depth O(log n)
  - note: size $\leq 2^{\text{depth}}$, so new formula has poly(n) size

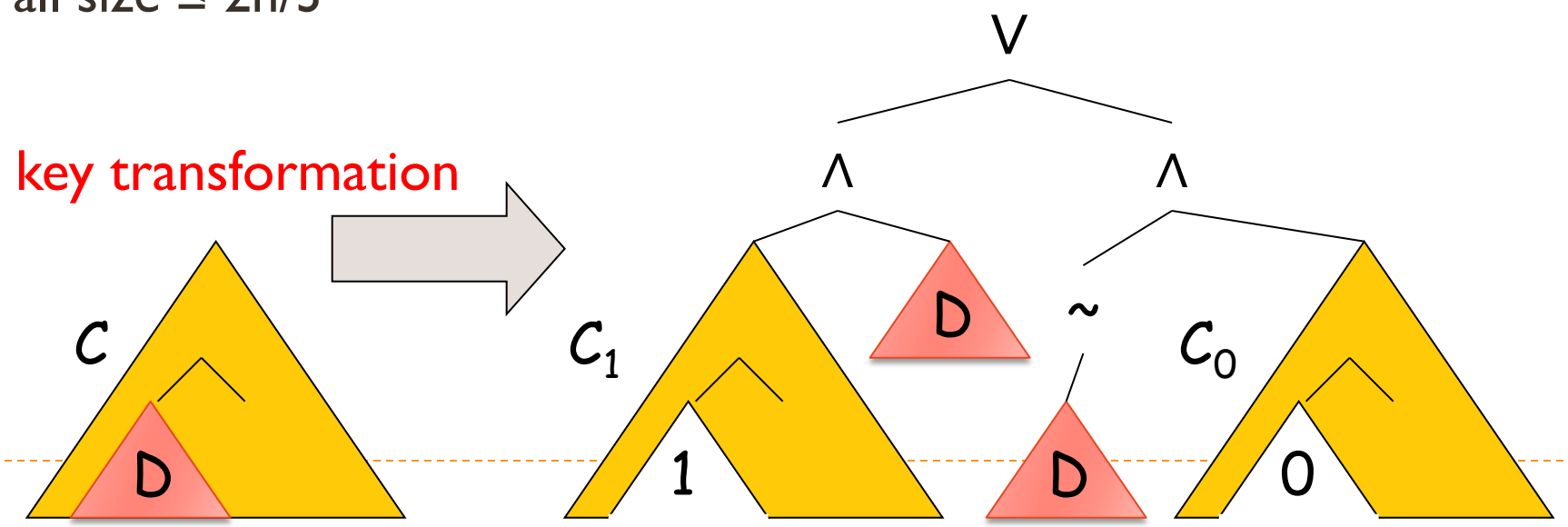


$\mathbf{NC^1}$ and Boolean formulas

- D is any minimal subtree with size ≥ n/3
  - implies size(D) < 2n/3

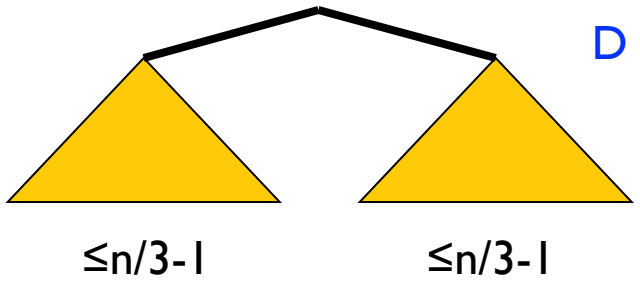


- define T(n) = maximum depth required for any size n formula
- C_1, C_0, D all size ≤ 2n/3



$\mathbf{NC^1}$ and Boolean formulas

- D is any minimal subtree with size ≥ n/3
  - implies size(D) < 2n/3



- define T(n) = maximum depth required for any size n formula
- C_1, C_0, D all size ≤ 2n/3

$$T(n) \leq T(2n/3) + 3$$

$$\text{implies } T(n) \leq O(\log n)$$



Relation to other classes

- Clearly **NC⊆P**
  - recall **P** = uniform poly-size circuits

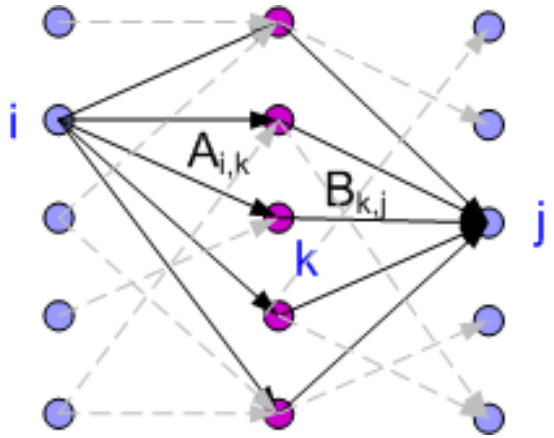
- **$NC^1 \subseteq L$**
  - on input x, compose logspace algorithms for:
    - generating $C_{|x|}$ with O(log n) depth
    - use DFS to evaluate the circuits $C_{|x|}(x)$
    - O(log n) depth so only logspace needed

  - Remember: If f and g are log-space computable functions, f∘g is also log-space computable



Relation to other classes

- **NL⊆NC²**: PATH∈ **NC²**

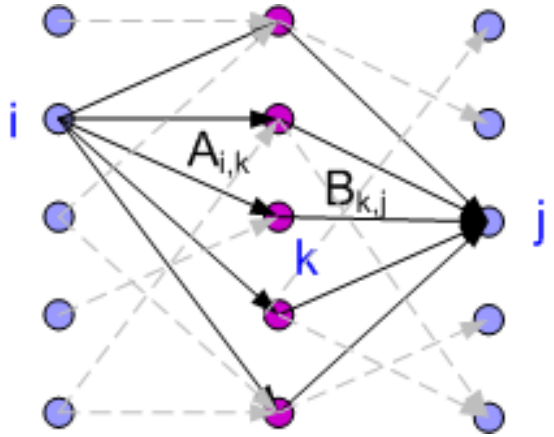




Relation to other classes

- **NL⊆NC²**: PATH∈ **NC²**
  - given G = (V, E), vertices s, t
  - A = adjacency matrix (with self-loops)
  - $(A^2)_{i,j}$ = 1 iff path of length ≤ 2 from node i to node j

  - Since $(A^2)_{i,j}=\vee_k(A_{i,k}\wedge A_{k,j})$
  - so if $(A^2)_{i,j}$= 1, there is a path (i,k,j) in the graph





Relation to other classes

- **NL⊆NC²**: PATH∈ **NC²**
  - given G = (V, E), vertices s, t
  - A = adjacency matrix (with self-loops)
  - $(A^2)_{i,j}$ = 1 iff path of length ≤ 2 from node i to node j
  - $(A^n)_{i,j}$ = 1 iff path of length ≤ n from node i to node j
  - compute with depth log n tree of Boolean matrix multiplications, output entry (s, t)
  - $\log^2 n$ depth in total



NC vs. P

- Can every efficient algorithm be efficiently parallelized?

$$\mathbf{NC} \stackrel{?}{=} \mathbf{P}$$

- **P**-complete problems least-likely to be parallelizable

- A language is **P**-complete if:
  - it is in **P**
  - every language in **P** is logspace-reducible to it

- if a **P**-complete problem is in **NC**, then **P** = **NC**
  - (homework)



Example of P-complete problem

- CIRCUIT-VALUE={<C,x>|C is a Boolean circuit and C(x)=1}

- CIRCUIT-VALUE is P-complete
  - By the proof of $P \subseteq P/poly$, reduce any <M,x> to <C,x>, where M is a polynomial time Turing machine



Example of P-complete problem

- CIRCUIT-VALUE={<C,x>|C is a Boolean circuit and C(x)=1}
- CIRCUIT-VALUE is P-complete
  - By the proof of $P \subseteq P/poly$, reduce any <M,x> to <C,x>, where M is a polynomial time Turing machine
- Linear Programming is also P-complete



NC Hierarchy Collapses

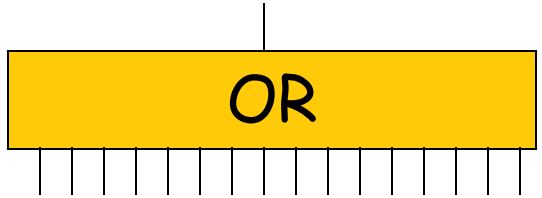
$$NC^1 \subseteq NC^2 \subseteq NC^3 \subseteq \ldots \subseteq NC$$

if $NC^i = NC^{i+1}$, then $NC = NC^i$



AC hierarchy *(alternation circuits)*

- AC^i is defined similarly to NC^i except gates are allowed unbounded fan-in
  - OR and AND gates can be applied to more than 2 bits

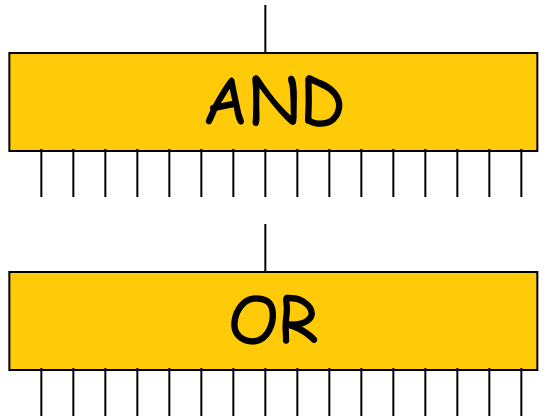


  - Such gates can be simulated by (log n)-depth gates of 2 bits
  - So $NC^i \subseteq AC^i \subseteq NC^{i+1}$



AC^0

- Then AC^0 is an interesting class, that is, computable by circuits:
  - with NOT, AND, OR gates, and AND, OR gates can have unbounded number of fan-in
  - polynomial number of gates s
  - constant depth d





Binary number addition

- Adding two numbers:

$$
\begin{array}{cccccc}
 & a_n & a_{n-1} & \cdots & a_1 & a_0 \\
+ & b_n & b_{n-1} & \cdots & b_1 & b_0 \\
\hline
r_{n+1} & r_n & r_{n-1} & \cdots & r_1 & r_0
\end{array}
$$

$$r_k = a_k \oplus b_k \oplus c_k$$
$$c_k = (c_{k-1} \wedge (a_k \vee b_k)) \vee (a_k \wedge b_k)$$



Need n depth to propagate the carry bit



Binary number addition

- Adding two numbers:

$$\begin{array}{cccccc} & a_n & a_{n-1} & \cdots & a_1 & a_0 \\ + & b_n & b_{n-1} & \cdots & b_1 & b_0 \\ \hline r_{n+1} & r_n & r_{n-1} & \cdots & r_1 & r_0 \end{array}$$

  - Notice that for c_1 to be 1, we must have $a_0 = b_0 = 1$
  - In general, for c_k to be 1, a carry bit has to be generated at some position $i < k$, and should be propagated all the way through to position k

$$c_k = \exists i (0 \leqslant i < k) \left[a_i \wedge b_i \bigwedge \forall j (i < j < k) (a_j \vee b_j) \right]$$



Binary number addition

- Adding two numbers:

$$\begin{array}{cccccc} & a_n & a_{n-1} & \cdots & a_1 & a_0 \\ + & b_n & b_{n-1} & \cdots & b_1 & b_0 \\ \hline r_{n+1} & r_n & r_{n-1} & \cdots & r_1 & r_0 \end{array}$$

- So $c_k = \bigvee_{0 \le i < k} \left(a_i \wedge b_i \wedge \bigwedge_{i < j < k} (a_j \vee b_j) \right)$

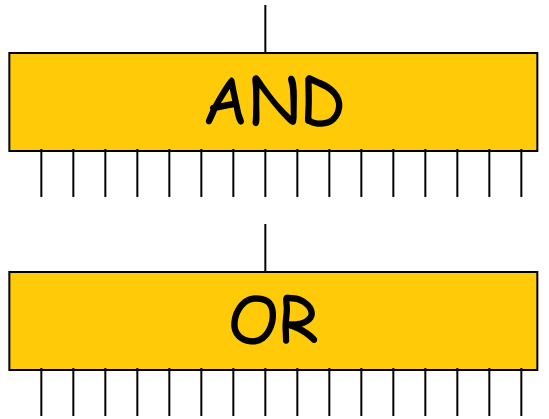
- So binary number addition is in AC^0



AC^0

- Then AC^0 is an interesting class, that is, computable by circuits:
  - with NOT, AND, OR gates, and AND, OR gates can have unbounded number of fan-in
  - polynomial number of gates s
  - constant depth d

- Any problem in NP not in AC^0?





- Parity function:

$$f(x_1,x_2,\ldots,x_n)=x_1 \oplus x_2 \oplus \ldots \oplus x_n$$

  - that is, f=1 iff an odd number of the inputs are 1

- Parity $\notin AC^0$ *[Furst, Saxe, Sipser 1981]*

Two ways to prove:
- Method of random restrictions
- Polynomial method



Polynomial Method

- Transfer constant depth circuits to low-degree polynomials

- Two statements:

1. Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$
2. Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

From 1 and 2 we can get Parity $\notin AC^0$ immediately



Polynomial Method

Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$

- Last lecture we use polynomial:

  $x_1 \vee x_2 \vee \ldots \vee x_n \rightarrow 1-(1-x_1)(1-x_2)\ldots(1-x_n)$

  - But the degree will be n

- We select $a_1,\ldots,a_n \in \mathbb{Z}_3$ at random, then

  $x_1 \vee x_2 \vee \ldots \vee x_n \rightarrow p_A=a_1x_1+a_2x_2+\ldots+a_nx_n$



Polynomial Method

Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$

- We select $a_1,\ldots,a_n \in \mathbb{Z}_3$ at random, then

  $$x_1 \vee x_2 \vee \ldots \vee x_n \rightarrow p_A = a_1x_1 + a_2x_2 + \ldots + a_nx_n$$

  - when $x_1 \vee x_2 \vee \ldots \vee x_n = 0$, $p_A = 0$
  - when $x_1 \vee x_2 \vee \ldots \vee x_n = 1$, $\Pr_A[p_A = 0] \leq 1/3$

- Error reduction?



Polynomial Method

Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$

- We select $a_1,\ldots,a_n \in \mathbb{Z}_3$ at random, then

  $$x_1 \vee x_2 \vee \ldots \vee x_n \rightarrow p_A = a_1x_1 + a_2x_2 + \ldots + a_nx_n$$

  - when $x_1 \vee x_2 \vee \ldots \vee x_n = 0$, $p_A = 0$
  - when $x_1 \vee x_2 \vee \ldots \vee x_n = 1$, $\Pr_A[p_A = 0] \leq 1/3$

- So we choose $\Theta(\log s)$ different random n-vectors $A_i=(a_{i1},\ldots,a_{in})$ and use $1 - \prod_i \left(1 - P_{A_i}^2\right)$
  - We can make the error prob. $<1/(100s)$



Polynomial Method

Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$



- So we choose $\approx\log_3(100s)$ different random n-vectors A_i, and use $1-\prod_i\left(1-P_{A_i}^2\right)$
  - so every OR gate changes to an O(log s)-degree polynomial
  - NOT x: 1-x
  - AND gate: by De Morgan's rule
- final degree: $(\log s)^{O(d)}$
- For every input, error prob.: <1%
  - every gate with error prob. <1/(100s)
- Then there is a set of vectors A so that the polynomial equals C for 99% of the input



Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

- Main idea:
  - We can transfer any polynomial from $\{0,1\}^n \to \{0,1\}$ to $\{-1,1\}^n \to \{-1,1\}$ with the same degree

$f(x_1,x_2,\ldots,x_n): \{0,1\}^n \to \{0,1\}$ is tranferred to:

$$g(x_1,x_2,\ldots,x_n)=2f\left(\frac{x_1+1}{2},\frac{x_2+1}{2},\ldots,\frac{x_n+1}{2}\right)-1$$

    - no degree increment



Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

- Main idea:
  - We can transfer any polynomial from $\{0,1\}^n \rightarrow \{0,1\}$ to $\{-1,1\}^n \rightarrow \{-1,1\}$ with the same degree

$f(x_1,x_2,\ldots,x_n): \{0,1\}^n \rightarrow \{0,1\}$ is tranferred to:

$g(x_1,x_2,\ldots,x_n) = -f(-x_1 - 1, -x_2 - 1, \ldots, -x_n - 1) - 1$ in $\mathbb{Z}_3$

    - no degree increment



Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

- Main idea:
  - We can transfer any polynomial from $\{0,1\}^n \to \{0,1\}$ to $\{-1,1\}^n \to \{-1,1\}$ with the same degree
  - then the Parity function defined on $\{-1,1\}^n \to \{-1,1\}$ is $x_1 \cdot x_2 \cdot \ldots \cdot x_n$
  - By contradictory assumption, $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$



Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

- Main idea:
  - We can transfer any polynomial from $\{0,1\}^n \to \{0,1\}$ to $\{-1,1\}^n \to \{-1,1\}$ with the same degree
  - then the Parity function defined on $\{-1,1\}^n \to \{-1,1\}$ is $x_1 \cdot x_2 \cdot \ldots \cdot x_n$
  - By contradictory assumption, $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$
  - In $\{-1,1\}^n$, $x_i = x_i^{-1}$, so

    $$x_1 \cdot x_3 \cdot x_4 \cdot x_7 = x_2 \cdot x_5 \cdot x_6 \cdot q(x_1, x_2, \ldots, x_7)$$

  - that is, monomial of degree d'>n/2 can be transferred to a degree (n-d') $+c \cdot n^{1/2}$ polynomial



Assume $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$

- Any function over $\{-1,1\}^n \rightarrow \{-1,1\}$ is equivalent to a polynomial of degree n in $\mathbb{Z}_3$
  - equivalent to a multi-linear polynomial, that is, sum of monomials in which the degree of every variable is 0 or 1
- Then any function over $S \rightarrow \{-1,1\}$ is equivalent to a polynomial of degree $\frac{n}{2} + c \cdot \sqrt{n}$

- In $\{-1,1\}^n$, $x_i = x_i^{-1}$, so
  $x_1 \cdot x_3 \cdot x_4 \cdot x_7 = x_2 \cdot x_5 \cdot x_6 \cdot q(x_1, x_2, \ldots, x_7)$
- that is, monomial of degree d'>n/2 can be transferred to a degree (n-d') $+c \cdot n^{1/2}$ polynomial



Assume $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$

- Function over $S \rightarrow \{-1,1\}$ is equivalent to:

- A polynomial of degree $\frac{n}{2} + c \cdot \sqrt{n}$ in $\mathbb{Z}_3$
  - equivalent to a multi-linear polynomial, that is, sum of monomials in which the degree of every variable is 0 or 1
  - so the coefficient can be -1, 0 or 1

We count the number of these two kinds of functions, and show the first kind has more than the second



Assume $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$

- Function over $S \rightarrow \{-1,1\}$ is equivalent to:

- A polynomial of degree $\frac{n}{2} + c \cdot \sqrt{n}$ in $\mathbb{Z}_3$
  - equivalent to a multi-linear polynomial, that is, sum of monomials in which the degree of every variable is 0 or 1
  - so the coefficient can be -1, 0 or 1

Number of monomials of degree k?
Total number of monomials?



Assume $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$

- Function over $S \rightarrow \{-1,1\}$ is equivalent to:

- A polynomial of degree $\frac{n}{2} + c \cdot \sqrt{n}$ in $\mathbb{Z}_3$
  - equivalent to a multi-linear polynomial, that is, sum of monomials in which the degree of every variable is 0 or 1
  - so the coefficient can be -1, 0 or 1

Number of monomials of degree k: $\binom{n}{k}$

Total number of monomials: $\sum_{i=0}^{\frac{n}{2}+c\cdot\sqrt{n}} \binom{n}{i}$



Assume $x_1 \cdot x_2 \cdot \ldots \cdot x_n = q(x_1, x_2, \ldots, x_n)$ for some polynomial q of degree $c \cdot n^{1/2}$ over a set $S \subseteq \{-1,1\}^n$ where $|S| \geq 0.99 \cdot 2^n$

- Function over S $\rightarrow \{-1,1\}$

  Number: $2^{0.99 \times 2^n}$

- A polynomial of degree $\frac{n}{2} + c\sqrt{n}$ in $\mathbb{Z}_3$

  Number: $3^{\sum_{i=0}^{\frac{n}{2}+c\cdot\sqrt{n}} \binom{n}{i}}$

  - we can choose c small, so that the former is larger than the latter



Polynomial Method

- Transfer constant depth circuits to low-degree polynomials

- Two statements:

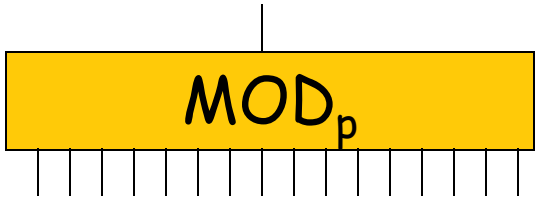
1. Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$
2. Parity cannot be 99% approximated by a polynomial of degree $c \cdot n^{1/2}$ over $\mathbb{Z}_3$

From 1 and 2 we can get Parity $\notin AC^0$ immediately



ACC (AC with counters)

- $AC^0[p]$: AC^0 with a module p counter
  - C=0 if $x_1+x_2+...+x_n \equiv 0 \pmod p$
  - C=1 otherwise



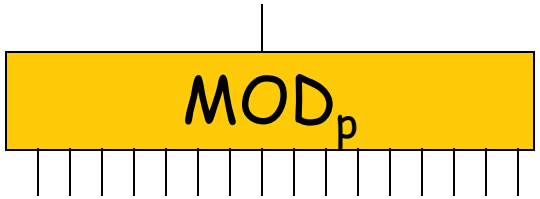
- ACC^0: U$AC^0[p]$

AC^0 cannot even compute Parity
What about we add a gate for Parity (SUM module 2)?



ACC (AC with counters)

- $AC^0[p]$: AC^0 with a module p counter
  - C=0 if $x_1+x_2+...+x_n \equiv 0 \pmod p$
  - C=1 otherwise



- ACC^0: $\cup AC^0[p]$

Aaronson says constant-depth ACC^0 is closely related to neural networks, as using gates AND, OR, XOR, THRESHOLD, MAJORITY…



- What we have proved:

  Parity $\notin AC^0[3]$

  - Since MOD_3 gate is just equivalent to $x_1+x_2+...+x_n$ in $\mathbb{Z}_3$



- What we have proved:

  Parity $\notin AC^0[3]$

- Generalized:

  Let p and q be distinct primes, then $MOD_q \notin AC^0[p]$



Lower bounds

- Recall: "**NP** does not have polynomial-size circuits" (**NP** $\not\subseteq$ **P/poly**) implies **P** $\neq$ **NP**
- **major goal**: prove lower bounds on (non-uniform) circuit size for problems in **NP**
  - believe exponential
  - super-polynomial enough for **P** $\neq$ **NP**
  - best bound known: linear
  - don't even have super-polynomial bounds for problems in **NEXP**



- [Williams 2010] NEXP $\not\subset$ ACC^0

Recall ACC^0 is $\cup_{p \geq 1} AC^0[p]$



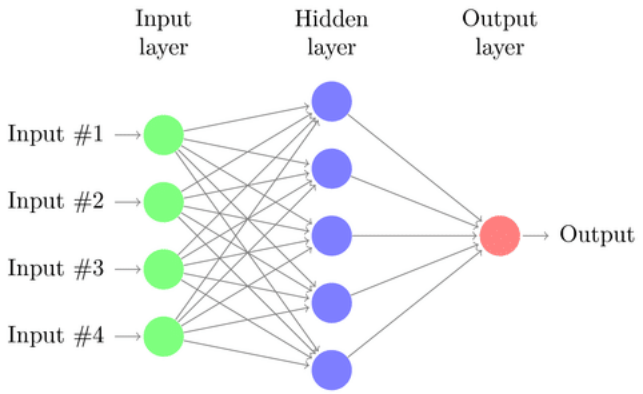
Why circuit complexity?

- Prove algorithm lower bound
  - Algorithms (Turing machines) are infinite
  - Circuits with n inputs has sizes $O(n2^n)$



- Related to circuit design?

- Neural networks are like circuits:
  - Many lower bounds for constant depth circuits





APSP with $n^3/2^{\Omega\left(\sqrt{\log n}\right)}$ time

- *[Ryan Williams 2014]* "Faster all-pairs shortest paths via circuit complexity"
  - works on real RAM
  - here we only consider the integer weights $[0, n^3]$
  - so the lengths of the numbers are O(log n)

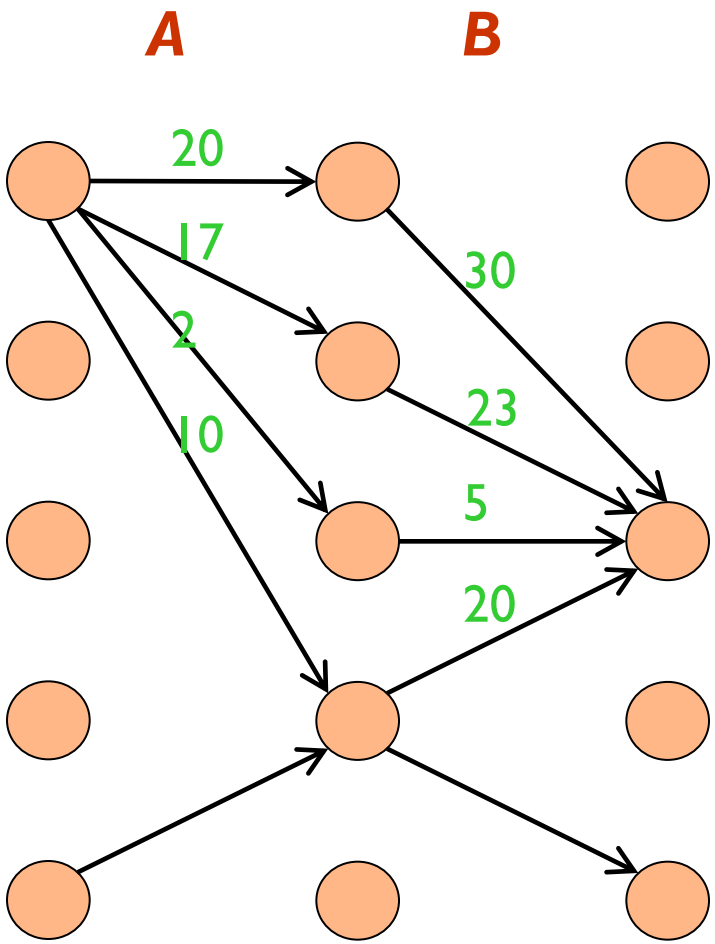


Progress on real-weighted APSP

Author	Runtime	Year
Fredman	$n^3 \log\log^{1/3} n / \log^{1/3} n$	1976
Takaoka	$n^3 \log\log^{1/2} n / \log^{1/2} n$	1992
Dobosiewicz	$n^3 / \log^{1/2} n$	1992
Han	$n^3 \log\log^{5/7} n / \log^{5/7} n$	2004
Takaoka	$n^3 \log\log^2 n / \log n$	2004
Zwick	$n^3 \log\log^{1/2} n / \log n$	2004
Chan	$n^3 / \log n$	2005
Han	$n^3 \log\log^{5/4} n / \log^{5/4} n$	2006
Chan	$n^3 \log\log^3 n / \log^2 n$	2007
Han, Takaoka	$n^3 \log\log n / \log^2 n$	2012
Williams	$n^3 / \exp(\sqrt{\log n})$	2014



An interesting special case of the APSP problem



$$C = A * B$$

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

Min-Plus product
/Distance product



Algebraic Products

$$C = AB$$

$$c_{ij} = \sum_k a_{ik} b_{kj}$$

Current bound is $O(n^{2.3714})$

Distance Products

$$C = A * B$$

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

Currently no $O(n^{2.999})$ time algorithm!



APSP $\Longrightarrow$ Distance product

If W is an n by n matrix containing the edge weights of a graph. Then W^n is the distance matrix

By induction, W^k gives the distances realized by paths that use at most k edges

$D \leftarrow W$
for $i \leftarrow 1$ to $\lceil \log_2 n \rceil$
    do $D \leftarrow D*D$

Thus: APSP(n) $\leq$ MPP(n) log n



- We want to compute distance product of two $n \times n$ matrices C=A*B:

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

- What about just two k-vectors u,v ?
  - $(u \circ v) = u^T * v = \min_k(u_k + v_k)$

- Circuits computing plus and minimum?



Circuits computing plus and minimum

- Adding two (log n)-bit numbers is in $(\log n)^{O(1)}$-size AC^0
- Computing the minimum of k numbers each with (log n)-bits can be done with poly(k, log n)-size AC^0

$$LEQ(x,y) = \left(\bigwedge_{i=1}^{\log n} (x_i = y_i) \vee \bigvee_{i=1}^{\log n} \left((x_i < y_i) \wedge \bigwedge_{j=1}^{i-1} (x_j = y_j) \right) \right)$$

(if $LEQ(x_1,x_2) \wedge LEQ(x_1,x_2) \wedge \ldots \wedge LEQ(x_1,x_k)$ output x_1)

- Thus, $(u \circ v) = \min_k(u_k + v_k)$ can be computed by poly(k, log n)-size AC^0 circuits

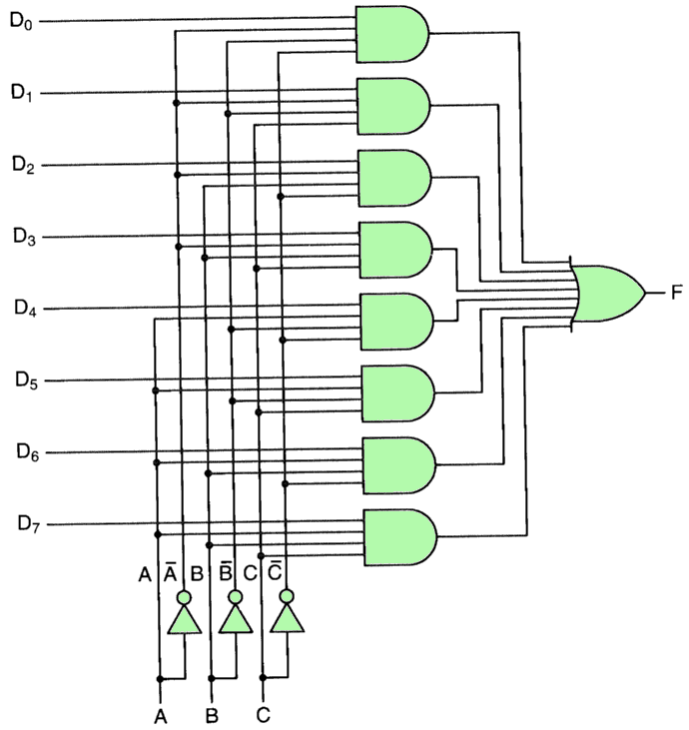


Multiplexers

- Multiplexer is also in AC^0

A	B	C	OUTPUT
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

n



log n

Polynomial Method

- Let C be an AC circuit of size s and depth d, then C is 99% approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_3=\{0,1,-1\}$

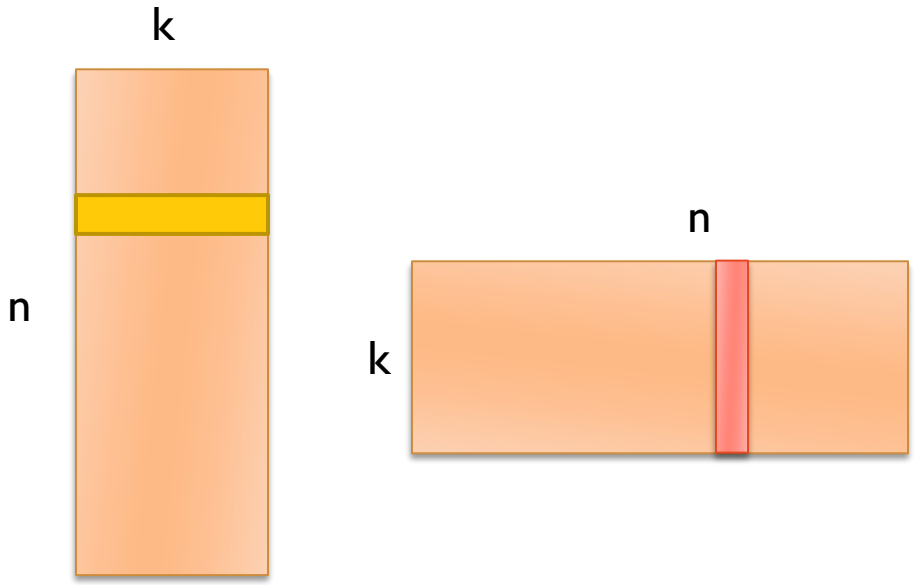


Polynomial Method

- Similarly, let C be an AC circuit of size s and depth d, then C is 99%-approximated by a polynomial of degree $(\log s)^{O(d)}$ over $\mathbb{Z}_2=\{0,1\}$
- Since $(u \circ v) = \min_k(u_k+v_k)$ for two k-vectors can computed by poly(k, log n)-size AC^0 circuit
- It can be 99%-approximated by a polynomial of degree $\log^{O(1)}(k \cdot \log n)$
  - input length $O(k \cdot \log n)$



- How can we compute the distance product of $n \times k$ and $k \times n$ matrices?
  - we need to compute all $n \times n$ pairs of $(u \circ v)$





- How can we compute the distance product of $n \times k$ and $k \times n$ matrices?
  - we need to compute all $n \times n$ pairs of $(u \circ v)$
  - Remember each $(u \circ v)$ has a polynomial of degree $D = \log^{O(1)}(k \cdot \log n)$
- The number of terms in a multi-linear polynomial of K inputs and degree D is $< K^D$
  - If this number is bounded by $n^{0.1}$…



All-pair "inner product"

- Two sets of vectors U,V, each u^i,v^j is a k-vector
  - we want to compute $(u^i \circ v^j)$ for all pairs of i and j

U: $\{u^1, u^2, \ldots, u^n\}$

V: $\{v^1, v^2, \ldots, v^n\}$

polynomial of degree D for each $(u^i \circ v^j)$:

$u^i_1 v^j_2 u^i_2 v^j_4 + u^i_2 v^j_5 u^i_3 v^j_1 + u^i_1 v^j_3 u^i_5 v^j_7 + \ldots + u^i_4 v^j_5 u^i_6 v^j_3$

$(k \cdot \log n)^D$ terms



All-pair "inner product"

- Two sets of vectors U,V, each u^i,v^i is a k-vector
  - we want to compute $(u^i \circ v^j)$ for all pairs of i and j

U: $\{u^1, u^2, \ldots, u^n\}$

V: $\{v^1, v^2, \ldots, v^n\}$

polynomial of degree D:

$u^i_1 v^j_2 u^i_2 v^j_4 + u^i_2 v^j_5 u^i_3 v^j_1 + u^i_1 v^j_3 u^i_5 v^j_7 + \ldots + u^i_4 v^j_5 u^i_6 v^j_3$

$(k \cdot \log n)^D$ terms

traditional inner product of

$(u^i_1 u^i_2, u^i_2 u^i_3, u^i_1 u^i_5, \ldots, u^i_4 u^i_6) \cdot (v^j_2 v^j_4, v^j_5 v^j_1, v^j_3 v^j_7, \ldots, v^j_5 v^j_3)$



All-pair "inner product"

- Two sets of vectors U,V, each u^i,v^i is a k-vector
  - we want to compute $(u^i \circ v^j)$ for all pairs of i and j

U: $\{u^1, u^2, \ldots, u^n\}$

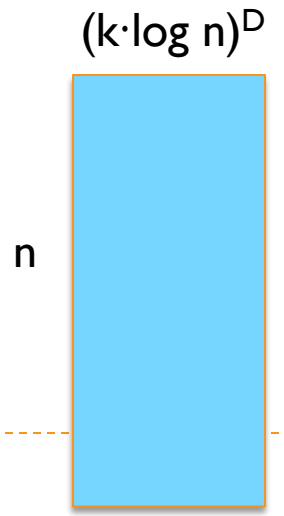
V: $\{v^1, v^2, \ldots, v^n\}$

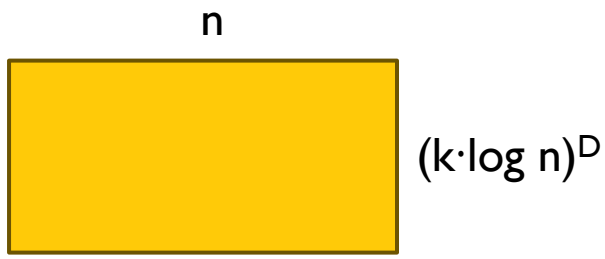
polynomial of degree D:

$u^i_1 v^j_2 u^i_2 v^j_4 + u^i_2 v^j_5 u^i_3 v^j_1 + u^i_1 v^j_3 u^i_5 v^j_7 + \ldots + u^i_4 v^j_5 u^i_6 v^j_3$

$(k \cdot \log n)^D$ terms

If $(k \cdot \log n)^D < n^{0.1}$, then FMM can compute the product of $n \times n^{0.1}$ and $n^{0.1} \times n$ matrices in $\tilde{O}(n^2)$ time





(traditional rectangular matrix multiplication)

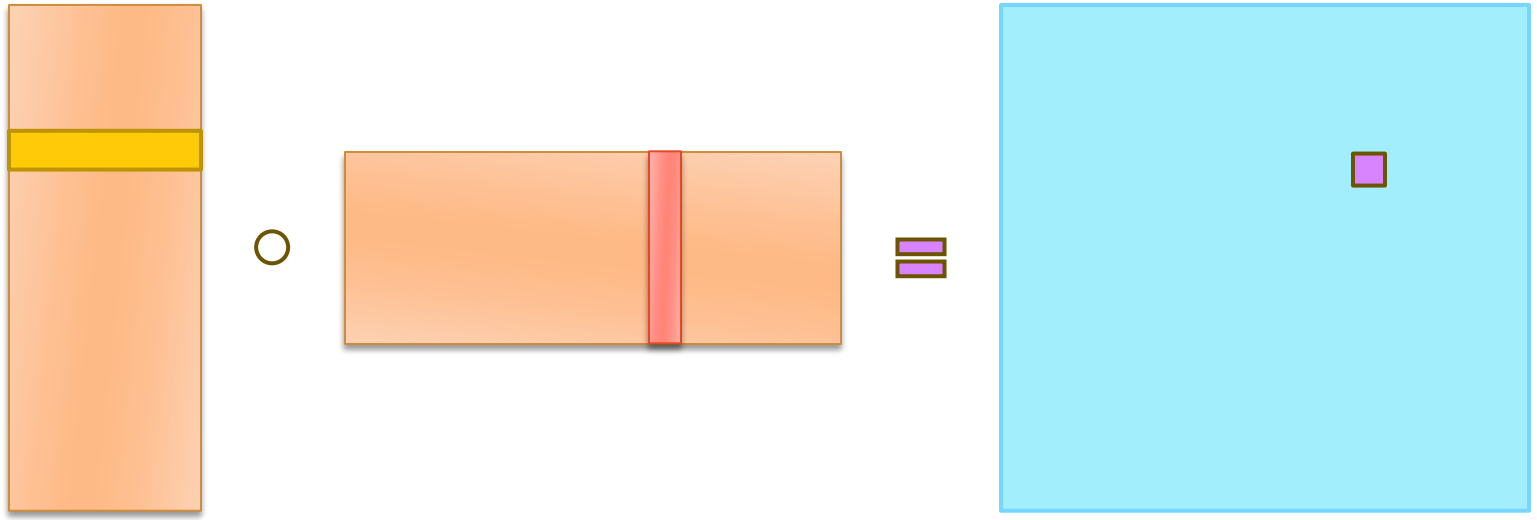
- How can we compute the distance product of n × k and k × n matrices?
  - we need to compute all n × n pairs of (u∘v)
  - Remember each (u∘v) has a polynomial of degree $D=\log^{O(1)}(k\cdot\log n)$
- The number of terms in a multi-linear polynomial of K inputs and degree D is $<K^D$
  - we want $(k\cdot\log n)^D< n^{0.1}$,

$$(k\cdot\log n)^{(\log(k\cdot\log n))^c}<n^{0.1}$$

- Let $k=2^{\Theta\left((\log n)^{1/(c+1)}\right)}$, then the distance product of n × k and k × n matrices can be computed in $\tilde{O}(n^2)$ time



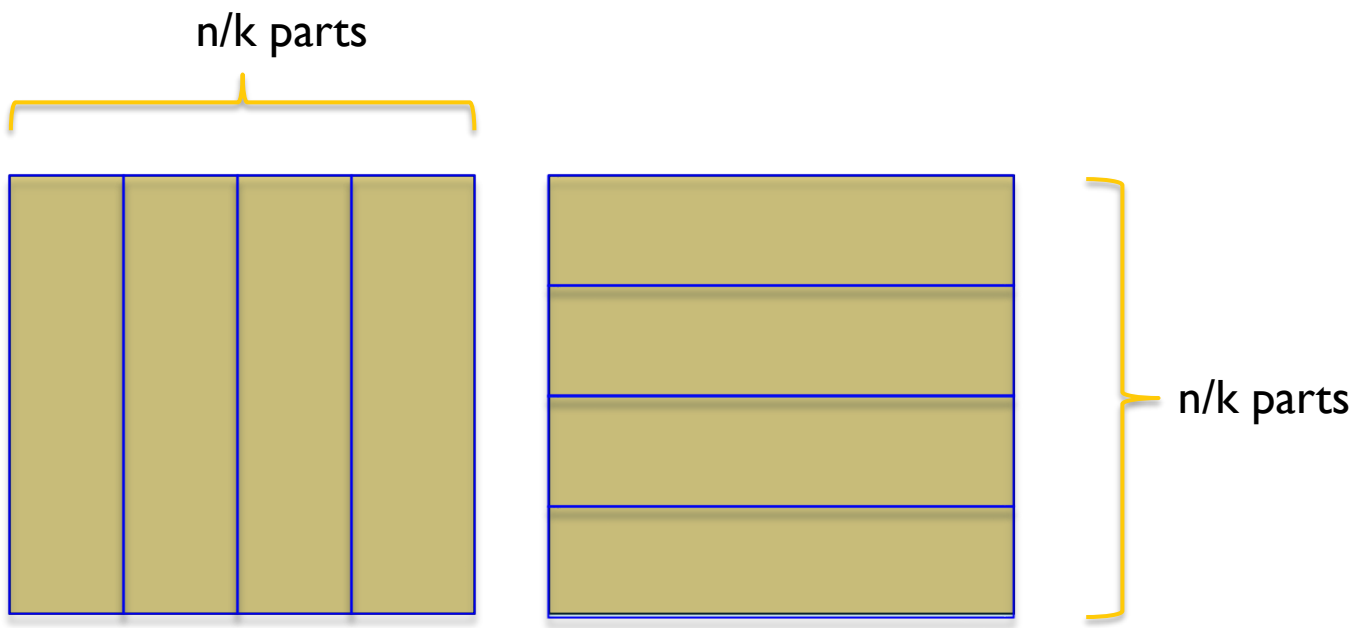
- Remember it is 99%-approximation of the circuits, so only 99% of $n \times n$ entries give correct answer
  - We can output index of the answer and check whether the answer exists
  - repeat the procedure O(log n) times to find the minimum answer for every entry





The algorithm

- Compute n/k distance products of n×k matrices and k×n matrices
- Total time: $\tilde{O}(n^3/k) = O\left(\frac{n^3}{2^{\Theta\left((\log n)^{1/(c+1)}\right)}}\right)$





Summary

- Circuit complexity
  - concepts of P/poly, formula, NC, AC, …
  - Karp-Lipton
  - P-completeness

- Polynomial method
  - proof of Parity$\notin AC^0$
  - faster APSP algorithm

